

Product Requirements Document (PRD) - WhatsApp-Controlled Multi-Agent Claude Code System

Version: 6.2

Author: Kshitij

Last Updated: April 2026

1. Overview

1.1 Product Summary

A **WhatsApp-controlled multi-agent development system** that remotely triggers Claude Code to execute coding tasks on any repository. Users send natural language instructions via WhatsApp, and a backend multi-agent system orchestrates planning, execution, debugging, and review using **Claude Code CLI as the execution engine**.

1.2 Vision

Enable a **fully remote, mobile-first development workflow** where:

- WhatsApp = control interface
 - Agents = orchestration logic
 - Claude Code = execution engine
-

2. Core Principle (Critical Design Shift)

Agents DO NOT call LLM APIs directly.

Agents orchestrate Claude Code CLI execution.

This ensures:

- Near-zero cost (uses Claude subscription)
 - Controlled execution
 - No runaway API usage
-

3. Goals

- Multi-agent orchestration from day one
 - WhatsApp-based control layer
 - Claude Code as primary execution engine
 - Support any repository dynamically
 - Deterministic workflows with cost control
-

4. System Architecture

```
WhatsApp
  ↓
Communication Agent
  ↓
Orchestration Agent (State Machine)
  ↓
Planning → Execution → Testing → Debugging → Review
  ↓
```

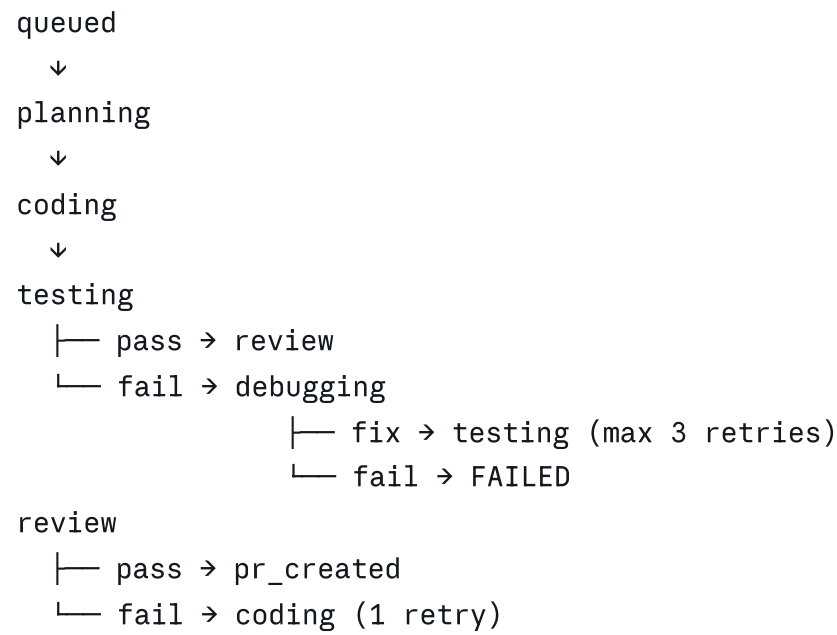
```
Claude Code CLI
  ↓
Repo Workspace (local filesystem)
  ↓
Git Operations
  ↓
Logging Agent
  ↓
Response → WhatsApp
```

5. Agents & Responsibilities

Agent	Role
Communication Agent	Handles WhatsApp messages via whatsapp-web.js , parses commands and sends responses
Orchestration Agent	Controls state machine and routing
Planning Agent	Breaks tasks into steps using Claude Code CLI planning prompts
Execution Agent	Triggers Claude Code CLI for coding + testing
Debugging Agent	Fixes errors via iterative Claude Code calls
Review Agent	Reviews code diffs via Claude Code with template: <u>Review the following code diff for bugs and style issues. Respond only with: PASS or FAIL and reasons.</u>

Agent	Role
Repo Agent	Manages repo cloning, switching, workspace isolation
Logging Agent	Tracks logs, state, outputs, and checkpoints

6. Orchestration (State Machine)



Rules

- Max retries per stage: 3
- Each transition logged
- Failures return control to user

7. Claude Code Execution Model

All execution happens via CLI:

```
bash  
  
cd /workspace/repo  
claude-code "<task>"
```

Usage by Agents

- **Execution Agent** → feature implementation
- **Debugging Agent** → fix errors
- **Planning Agent** → break down tasks
- **Review Agent** → review diffs for PASS/FAIL

8. Token Budget System

Each task enforces execution limits:

```
json  
  
{  
  "max_invocations": 15,  
  "current_invocations": 0  
}
```

Behavior

- Increment per Claude Code call
 - If limit reached:
 - Task pauses
 - User notified via WhatsApp
 - User replies `/resume` (grants 10 more) or `/cancel`
-

9. Messaging Abstraction Layer & Interface Contract

```
MessagingAbstractionLayer {  
    connect()  
    disconnect()  
    onMessage(callback)  
    sendMessage(user, message)  
    getStatus()  
}
```

Reliability

- Heartbeat every 60s
- Message retry: 3 times per delivery
- On disconnect: pause tasks, persist state
- On reconnect: resume from checkpoint
- Acknowledgment latency (task accepted): <2s
- First substantive response: <5s

10. Repo Management & Isolation

Workspace

```
workspace/  
├─ repo_1/  
├─ repo_2/
```

Rules

- One active task per repo (MVP)
 - Prevent concurrent writes
 - Each repo has isolated directory
-

11. Agent Communication Protocol

Event-driven architecture with shared state.

Task Object

```
json
```

```
{  
  "task_id": "uuid",  
  "status": "coding",  
  "repo": "repo_1",  
  "invocations": 5  
}
```

Events

```
json  
  
{  
  "event": "test_failed",  
  "task_id": "123"  
}
```

- Agents emit events
 - Orchestrator routes next step
-

12. Functional Requirements

Messaging

- Natural language + slash commands
- Task accepted acknowledgment: <2s
- First substantive response: <5s
- Only pre-registered WhatsApp numbers allowed

Repo Management

- `/connect <repo>` — clone and activate
- `/switch <repo>` — change active workspace
- Persistent workspace across sessions

Execution

- Claude Code-driven workflows
- Retries + debugging loops
- Planning and Review Agents use Claude Code CLI

Logging

- Structured logs
 - Checkpointed every 2 min (captures task status, current stage, invocation count, and last successful file state)
 - Accessible via `/logs`
-

13. Non-Functional Requirements

- Async execution with first response latency <5s
- Persistent task state
- Heartbeat: 60s
- Message retry: 3x per delivery
- Task retry limits: 3 per stage

- Checkpoint interval: 2 min (captures task status, current stage, invocation count, last successful file state)
 - Logging flush interval: 30s
 - Safe file handling: `.env` blocked, repo isolation enforced
-

14. Constraints & Risks

WhatsApp Instability

- Abstraction layer enables provider swap (Telegram, Slack)
- Heartbeat + auto-pause prevents silent failures

Execution Errors

- Retry limits (3 per stage)
- Human-in-the-loop for unrecoverable failures

Security

- Whitelist-based authentication (pre-registered numbers only)
- Ignore unauthorized messages silently

Cost Control

- Agents never call LLM APIs directly
 - Token budget enforced per task
-

15. MVP Scope

Included

- WhatsApp bot (whatsapp-web.js)
- Full multi-agent system (8 agents)
- Claude Code CLI execution
- State machine orchestration
- Repo management with workspace isolation
- Token budget system
- Persistent task storage + checkpoint resume

Excluded (Phase 2+)

- Multi-user support
- Distributed scaling

16. Success Metrics

Metric	Target
Task success rate	>70%
Avg retries	<2
Task time	<15 min

Metric	Target
Cost per task	Near-zero

17. Development Plan

Phase 1

- WhatsApp bot via whatsapp-web.js
- Repo Agent
- Claude Code execution
- Messaging Abstraction Layer
- Authentication (pre-registered number only)

Phase 2

- Orchestrator (state machine)
- Planning Agent with Claude Code CLI
- Debugging loops

Phase 3

- Review + Logging agents
- Token budget enforcement
- Persistent task storage & startup recovery
- Heartbeat & disconnect handling

18. Key Design Principles

1. Claude Code is the execution engine
 2. Agents are orchestration logic only
 3. No heavy API dependency
 4. Deterministic workflows
 5. Human-in-the-loop safety
-

19. Product Positioning

┆ "A WhatsApp-controlled multi-agent AI developer powered by Claude Code"

20. Appendix: Sample Interaction Flows

A. Happy Path

```
User: Add CSV export
System: Task accepted. Starting planner. Budget: 0/15.
System: Planning complete (3 steps). Starting coding. Budget: 2/15.
System: Coding complete. Running tests. Budget: 5/15.
System: Tests passed (4/4). Starting review. Budget: 6/15.
System: Review: PASS. Creating PR. Budget: 7/15.
System: PR created.
      Branch: feature/csv-export
      Tests: 4/4 passed
```

Files changed: 3
Budget used: 7/15
Link: github.com/repo/pull/42

B. Debugging Loop

User: Add CSV export
System: Task accepted. Starting planner. Budget: 0/15.
System: Planning complete. Starting coding. Budget: 2/15.
System: Coding complete. Running tests. Budget: 5/15.
System: Tests failed (1/4). Starting debug (1/3). Budget: 6/15.
System: Patch applied. Rerunning tests. Budget: 7/15.
System: Tests passed (4/4). Starting review. Budget: 8/15.
System: Review: PASS. PR created. Budget: 9/15.

C. Max Retries Exhausted

User: Add CSV export
System: Task accepted. Budget: 0/15.
System: Tests failed. Debug attempt 1/3. Budget: 6/15.
System: Tests failed. Debug attempt 2/3. Budget: 8/15.
System: Tests failed. Debug attempt 3/3. Budget: 10/15.
System: Task FAILED after 3 attempts.
Error: TypeError in csv_parser.py line 42
Last action: added pandas import
Reply /resume to retry, or /logs for full trace.

D. Budget Exceeded

System: Task hit execution limit (15/15).

```
Stage: Debugging (attempt 2/3)
Reply /resume to grant 10 more, or /cancel to abort.
User: /resume
System: Budget extended to 25. Resuming debug attempt 2/3.
```

E. Session Disconnect

```
System (via fallback): WhatsApp session disconnected.
Active task "CSV export" paused at stage: coding (Budget: 5/15).
Reconnect to resume.
[after reconnect]
System: Session restored. Resuming task "CSV export" from coding stage.
Budget: 5/15.
```

F. Repo Management

```
User: /connect https://github.com/user/analytics-app
System: Cloned analytics-app. Set as active workspace.
User: /switch other-repo
System: Switched to other-repo. Previous task on analytics-app preserved.
```

End of Document